FRONTEND / 2026

A more interactive Apex Motors.

Assignment 5: Advanced JavaScript and DOM Manipulation

Team: Apex Motors Group: SE-2507
Members: Arsen, Kuttibay, Farabi, Muhammad
Prepared: 28 October 2026

Objective. Continue the existing dealership project with meaningful DOM updates, event handling, reusable data structures, callbacks, sound and animation. The result is a responsive collection browser and a three-step booking preview.

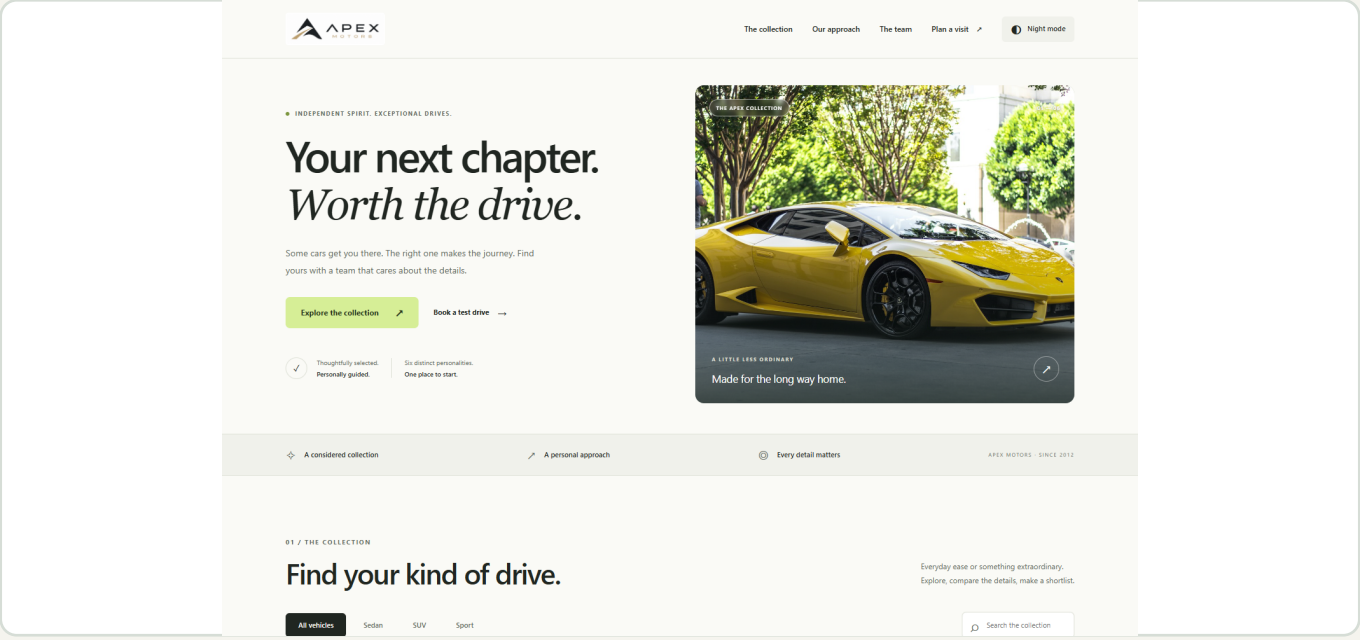


Figure 1. The completed Apex Motors website, using the team's existing visual theme and local images.

Published website
https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/

Source repository
https://github.com/Vertex-blip/Car-dealership-website

COVERAGE / COMPLETE CHECKLIST

Requirements made concrete.

The brief offers alternatives in several tasks. This project selects the day/night switch, image gallery, multi-step form and category filter. Story, stars, FAQ and a clock provide additional interactions. No network POST is required for the chosen callback option.

DOM and styling / 10% Selectors; dynamic text; image src/alt; story visibility; theme; stars.	3-6
Event handling / 10% Click/input/change listeners; arrow menu navigation; callback form; switch.	4, 7-9
Data and functions / 10% Vehicle objects and methods; arrays; explicit loop; filter/forEach callbacks.	3-4
Sound / 5% User-triggered local notification; single playback prevents overlapping.	10
Animation / 5% Event-triggered transforms and CSS transitions; reduced-motion support.	10
Code structure / 5% One HTML document, external CSS and four deferred JavaScript files.	11
UX / 5% Responsive collection, empty states, clear validation, dialog and keyboard focus.	3-11
Defense / 40% Each student explains the implementation and completes the instructor's task.	12 + guide
Report / 10% Objective, team/group, steps, actual code and page evidence, URL, reflection.	All pages

Submission package: PDF report and ZIP with HTML, CSS, JavaScript, local assets and evidence. All four team members submit individually and defend the work.

01 / IMPLEMENTATION EVIDENCE

Data becomes a collection.

Vehicle information is structured once, then rendered into real DOM elements. The fictional listing names and specifications are clearly presented as demonstration data.

1. Create vehicle objects with properties and a summary() method; store six objects in an array.
2. Select the collection container, create each card with a reusable function, and loop over the array.
3. Insert text with textContent; use filter and callbacks for derived collections.

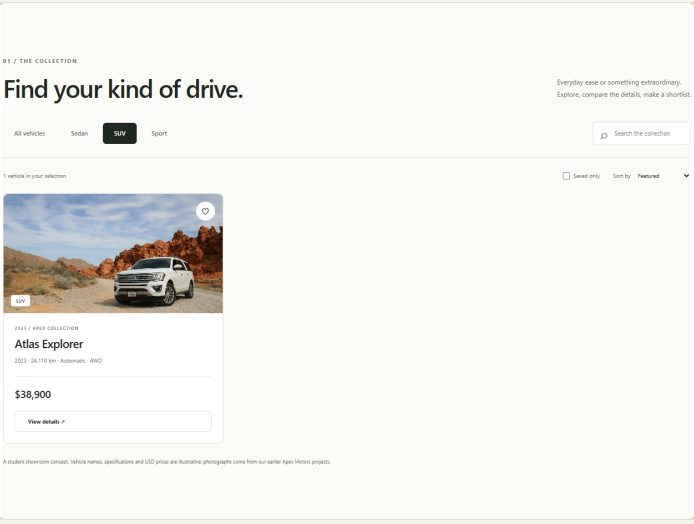


Figure 2. Dynamic vehicle cards and the live result count.

js/data.js

```
  5    function createVehicle(id, name, category, year, price, distance, drive, image,
     description) {
  6      return {
  7        id, name, category, year, price, distance, drive, image, description,
  8        // An object method keeps presentation logic beside the properties it describes.
  9        summary() {
 10          return `${this.year} · ${this.distance.toLocaleString('en-US')} km · ${this.drive}`;
 11        }
 12      };
 13    }
```

js/inventory.js

```
 78      const filtered = vehicles.filter(vehicle => matchesCategory(vehicle)
 79        && (!savedOnly.checked || saved.has(vehicle.id))
 80        && `${vehicle.name} ${vehicle.category} ${vehicle.year}
     ${vehicle.description}`.toLowerCase().includes(query));
 81      if (sort.value === 'price-asc') filtered.sort((a, b) => a.price - b.price);
 82      if (sort.value === 'price-desc') filtered.sort((a, b) => b.price - a.price);
 83      const fragment = document.createDocumentFragment();
 84      // Explicit loop creates one article per vehicle without inserting user-written HTML.
 85      for (const vehicle of filtered) fragment.append(makeCard(vehicle));
 86      grid.replaceChildren(fragment);
 87      count.textContent = `${filtered.length} ${filtered.length === 1 ? 'vehicle' :
     'vehicles'} in your selection`;
```

Actual source excerpt: js/data.js, js/inventory.js.

02 / IMPLEMENTATION EVIDENCE

Find the right vehicle.

Category, search, saved-only and price sorting work together without reloading the page. The displayed count, button states and empty state reflect the current selection.

1. Capture category button clicks and use switch to choose the actual filtering branch.
2. Combine the category predicate with the search and saved predicates; sort the resulting array.
3. Save a vehicle, update its pressed state and counter, and reset all filters with one button.

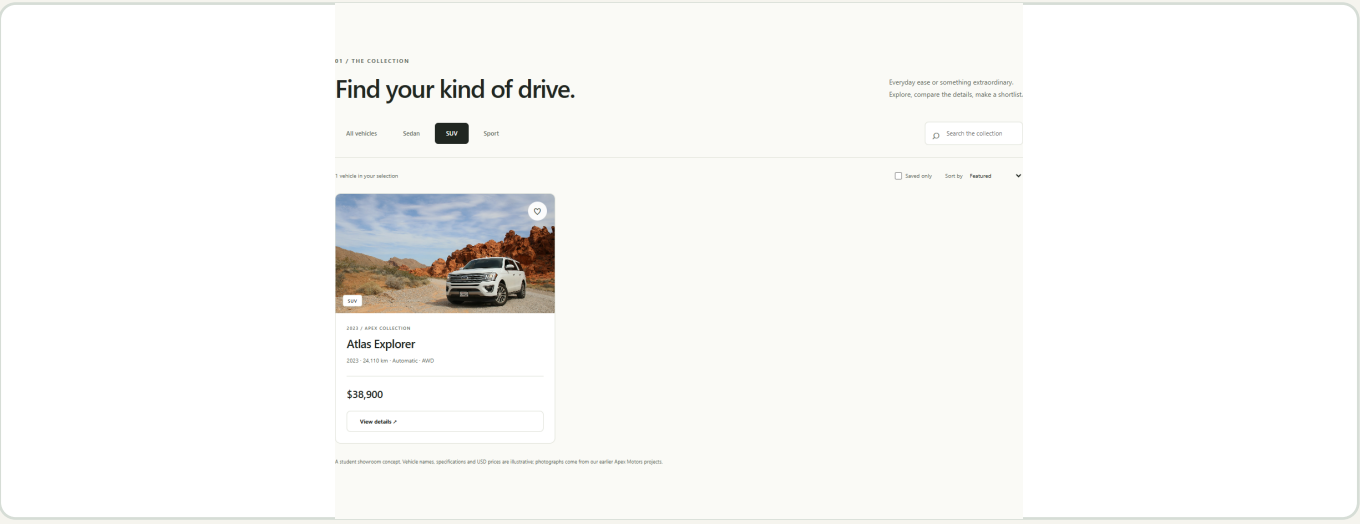


Figure 3. A selected category changes the results in place.

```
js/inventory.js
 32    function matchesCategory(vehicle) {
 33      // The switch controls the actual filtering behavior, rather than decorative labels.
 34      switch (category) {
 35        case 'sport': return vehicle.category === 'sport';
 36        case 'suv': return vehicle.category === 'suv';
 37        case 'sedan': return vehicle.category === 'sedan';
 38        default: return true;
 39      }
 40    }
 41
js/inventory.js
 94    filterButtons.forEach(button => button.addEventListener('click', () => {
 95      category = button.dataset.filter;
 96      filterButtons.forEach(item => {
 97        const active = item === button;
 98        item.classList.toggle('is-active', active);
 99        item.setAttribute('aria-pressed', String(active));
100      });
101      render();
102    }));
103    search.addEventListener('input', () => render());
```

Actual source excerpt: js/inventory.js.

03 / IMPLEMENTATION EVIDENCE

Attributes tell the story.

The detail gallery fulfills the attribute-manipulation choice. Additional dealership content can also be revealed with a Read more button.

1. Open vehicle details and build thumbnail buttons from a small gallery array.
2. On thumbnail click, update the large image's src and alt, caption and selected state.
3. Toggle additional story content through style.display and keep aria-expanded synchronized.

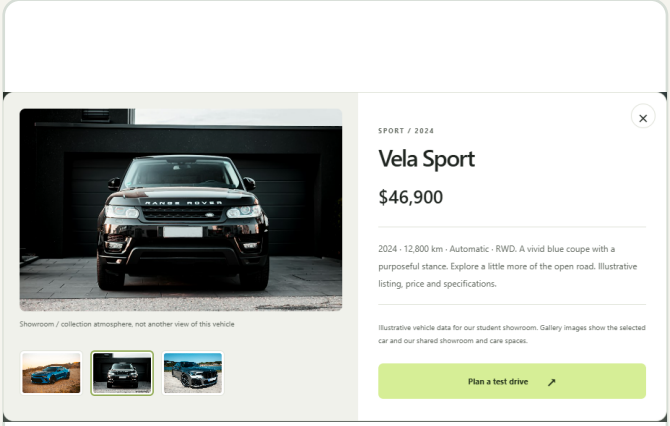


Figure 4. Selected thumbnail updates the gallery image.

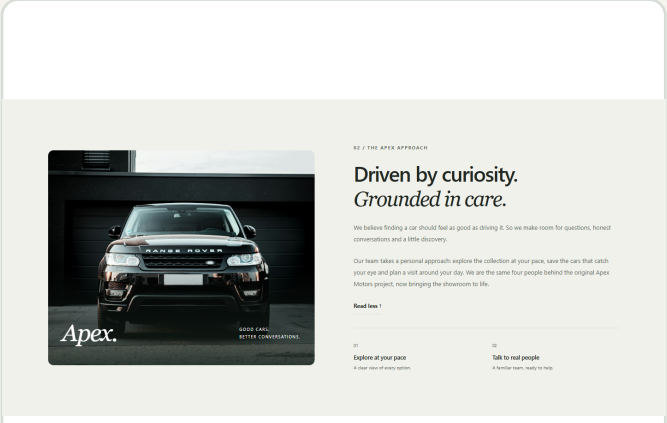


Figure 5. Read more reveals the dealership story.

js/inventory.js

```
118    function selectGalleryImage(item, button) {
119      const image = document.getElementById('detail-image');
120      image.src = item.src;
121      image.alt = item.alt;
122      document.getElementById('detail-caption').textContent = item.caption;
123      document.querySelectorAll('#detail-thumbnails button').forEach(thumbnail => {
124        thumbnail.classList.toggle('is-active', thumbnail === button);
125        thumbnail.setAttribute('aria-pressed', String(thumbnail === button));
126      });
127      window.Apex.animate(image);
128    }
129
```

js/experience.js

```
 68    readMore.addEventListener('click', () => {
 69      const open = readMore.getAttribute('aria-expanded') !== 'true';
 70      extra.style.display = open ? 'block' : 'none';
 71      readMore.setAttribute('aria-expanded', String(open));
 72      readMore.textContent = open ? 'Read less ↑' : 'Read our story □';
 73      if (open) window.Apex.animate(extra);
 74    });
 75
```

Actual source excerpt: js/inventory.js, js/experience.js.

04 / IMPLEMENTATION EVIDENCE

A personal atmosphere.

The day/night switch changes the page appearance consistently. Rating stars demonstrate selecting a group of elements and updating their visual and accessible state.

1. Listen for the theme button click, change the root theme attribute and update the control label.
2. Use querySelectorAll to collect the five star buttons and attach click listeners.
3. Update selected classes, aria-pressed attributes and rating feedback; allow the rating to reset.

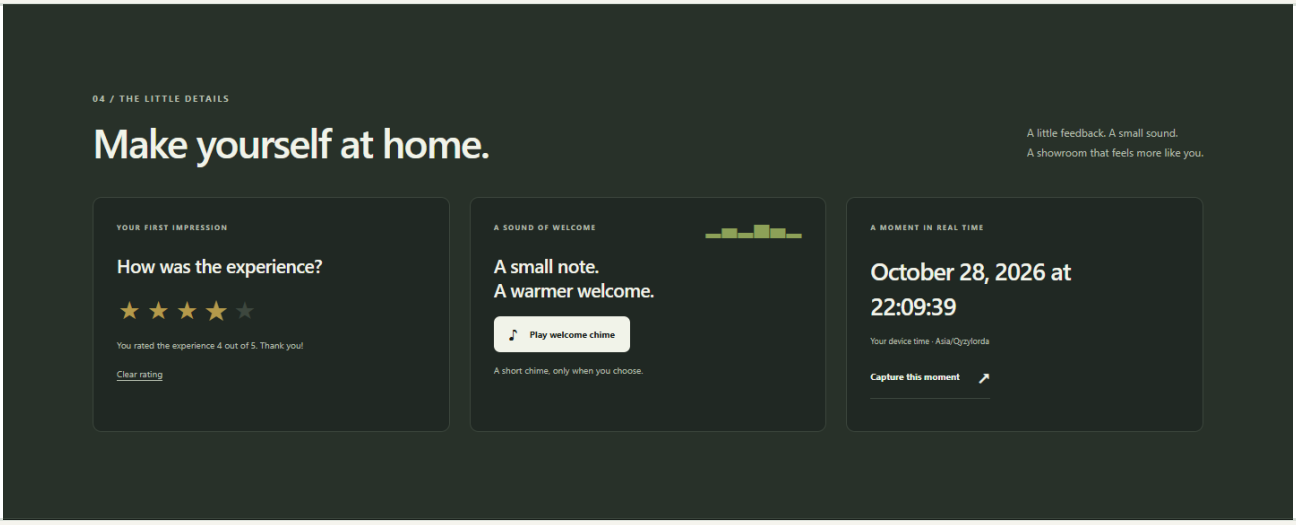


Figure 6. Night theme and selected star rating.

js/experience.js

```
  9    function applyTheme(theme) {
 10      const dark = theme === 'dark';
 11      root.dataset.theme = dark ? 'dark' : 'light';
 12      themeButton.setAttribute('aria-pressed', String(dark));
 13      themeLabel.textContent = dark ? 'Day mode' : 'Night mode';
 14      themeButton.setAttribute('aria-label', `Switch to ${dark ? 'day' : 'night'} mode`);
 15    }
 16    let savedTheme = 'light';
 17    try { savedTheme = localStorage.getItem('apex-theme') || 'light'; } catch { /* Optional
    persistence. */ }
 18    applyTheme(savedTheme);
```

js/experience.js

```
 79    function showRating(value) {
 80      selectedRating = value;
 81      stars.forEach(star => {
 82        const selected = Number(star.dataset.rating) <= selectedRating;
 83        star.classList.toggle('is-selected', selected);
 84        star.setAttribute('aria-pressed', String(Number(star.dataset.rating) ===
    selectedRating));
 85      });
 86      ratingStatus.textContent = selectedRating ? `You rated the experience ${selectedRating}
    out of 5. Thank you!` : 'How was your visit? Choose a star.';
 87    }
 88    stars.forEach(star => star.addEventListener('click', () => {
 89      showRating(Number(star.dataset.rating));
```

Actual source excerpt: js/experience.js.

05 / IMPLEMENTATION EVIDENCE

Navigation from the keyboard.

The navigation remains a normal set of links and supports the assignment's required arrow-key focus movement. Mobile navigation can be opened and closed with its labeled button.

1. Collect navigation links and register a keydown listener on the navigation container.
2. Detect arrows, Home and End; compute the destination with wraparound and prevent page scrolling.
3. Call focus() on the destination. Tab and Enter retain their normal browser behavior.

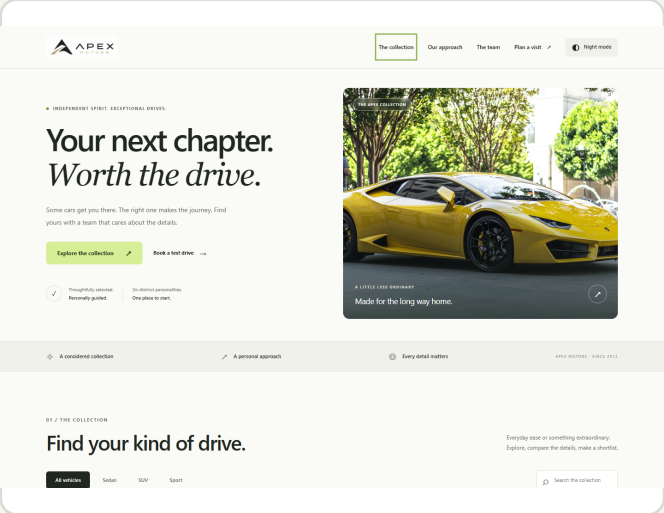


Figure 7. A navigation link receives keyboard focus.

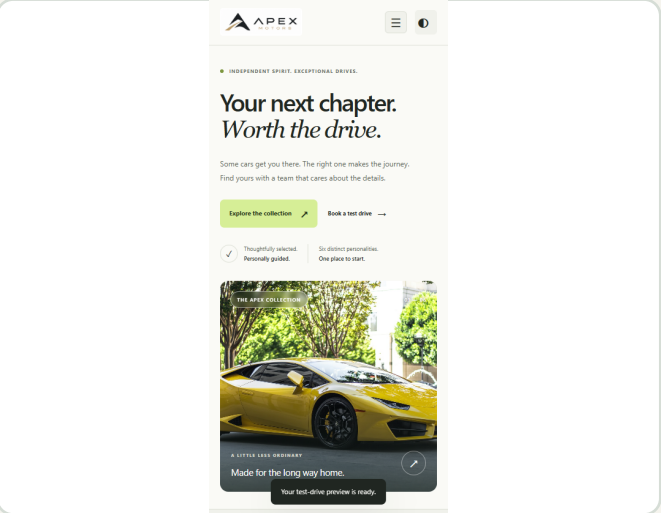


Figure 8. The same site adapts to a narrow mobile viewport.

js/experience.js

```
39    nav.addEventListener('keydown', event => {
40      const index = navLinks.indexOf(document.activeElement);
41      if (index < 0) return;
42      let next;
43      switch (event.key) {
44        case 'ArrowRight':
45        case 'ArrowDown': next = (index + 1) % navLinks.length; break;
46        case 'ArrowLeft':
47        case 'ArrowUp': next = (index - 1 + navLinks.length) % navLinks.length; break;
48        case 'Home': next = 0; break;
49        case 'End': next = navLinks.length - 1; break;
50        case 'Escape':
51          if (nav.classList.contains('is-open') && window.matchMedia('(max-width:
    640px)').matches) {
52            closeMenu();
53            menuButton.focus();
54          }
55          return;
56        default: return;
57      }
58      event.preventDefault();
59      navLinks[next].focus();
60    });
```

Actual source excerpt: js/experience.js.

06 / IMPLEMENTATION EVIDENCE

One form, three steps.

The callback requirement is implemented with a multi-step form. The step changes while the same input elements remain in the DOM, so going Back preserves entered values.

1. Keep the current step in state and validate the visible step before moving forward.
2. Pass a display callback to the navigation function; it changes fieldset visibility and progress.
3. Read the same fields to build the review summary. Back changes the step without clearing inputs.

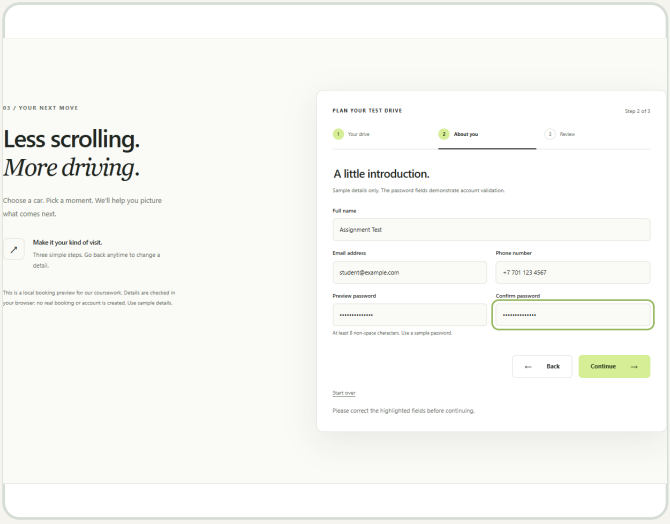


Figure 9. Contact details on the second step.

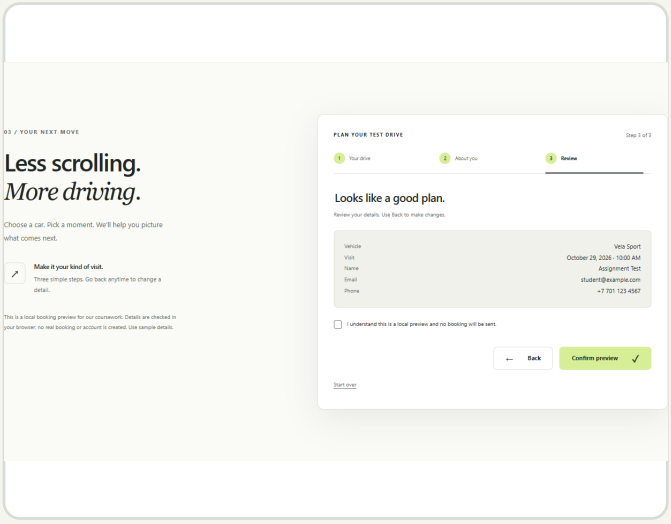


Figure 10. Review before confirmation.

```
js/booking.js
128    function navigate(direction, onStepChange) {
129      if (direction > 0 && !validateStep(currentStep)) {
130        status.textContent = 'Please correct the highlighted fields before continuing.';
131        return;
132      }
133      const nextStep = Math.min(2, Math.max(0, currentStep + direction));
134      onStepChange(nextStep);
135    }
136    nextButton.addEventListener('click', () => navigate(1, showStep));
137    backButton.addEventListener('click', () => navigate(-1, showStep));
138
js/booking.js
103    function showStep(index, focus = true) {
104      currentStep = index;
105      steps.forEach((step, position) => { step.hidden = position !== currentStep; });
106      document.querySelectorAll('[data-step-indicator]').forEach(indicator => {
107        const index = Number(indicator.dataset.stepIndicator);
108        indicator.classList.toggle('is-active', index === currentStep);
109        indicator.classList.toggle('is-complete', index < currentStep);
110        if (index === currentStep) indicator.setAttribute('aria-current', 'step');
```

Actual source excerpt: js/booking.js.

07 / IMPLEMENTATION EVIDENCE

Helpful validation and feedback.

The booking preview checks required inputs, email, date, phone, password length and confirmation. It gives inline feedback and does not send personal information to a server.

1. Prevent default form submission; check the step's fields and move focus to the first invalid input.
2. Revalidate before confirmation, require consent, and display a local success summary.
3. Clear sensitive fields after success. Reset or Book again clears values, errors and step state.

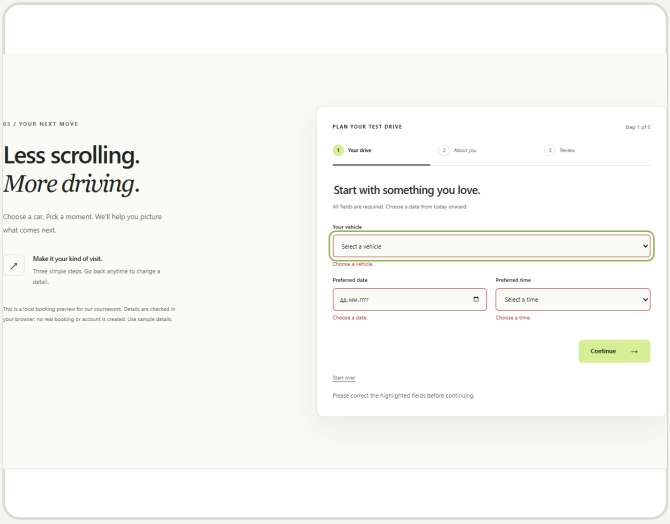


Figure 11. Missing or invalid fields receive clear errors.

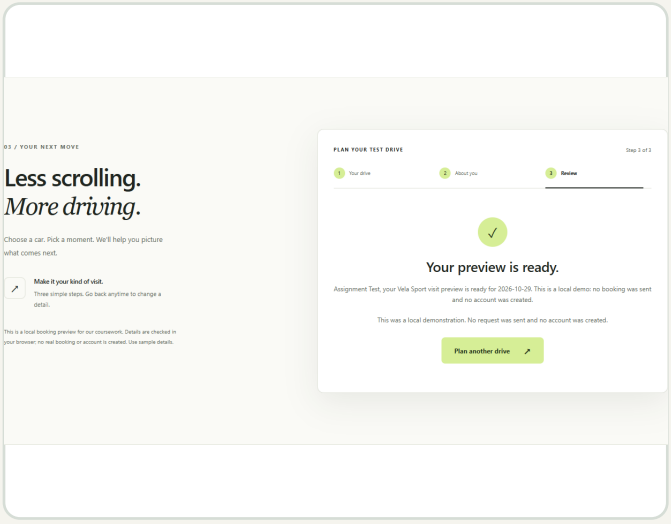


Figure 12. Successful local booking preview.

js/booking.js

```
 67    function validateStep(index, focusFirst = true) {
 68      let firstInvalid = null;
 69      for (const id of fieldsByStep[index]) {
 70        const valid = setError(id, fieldError(id));
 71        if (!valid && !firstInvalid) firstInvalid = document.getElementById(id);
 72      }
 73      if (firstInvalid && focusFirst) firstInvalid.focus();
 74      return !firstInvalid;
 75    }
```

js/booking.js

```
139    function resetBooking() {
140      form.reset();
141      // All input values, including hidden-step passwords, are explicitly cleared.
142      form.querySelectorAll('input').forEach(input => {
143        if (input.type === 'checkbox') input.checked = false;
144        else input.value = '';
145      });
146      vehicleSelect.value = '';
147      document.getElementById('booking-time').value = '';
148      fieldsByStep.flat().forEach(id => setError(id, ''));
```

Actual source excerpt: js/booking.js.

08 / IMPLEMENTATION EVIDENCE

Sound and movement with purpose.

Sound is triggered only by an intentional click. A single reusable audio instance is restarted for each click, and animations respond to actions rather than running continuously.

1. Load the bundled notification sound and stop/rewind previous playback before starting it again.
2. Update the sound feedback after playback starts; handle unsupported or blocked audio cleanly.
3. Animate result, gallery and step changes with opacity/transform; respect reduced-motion settings.

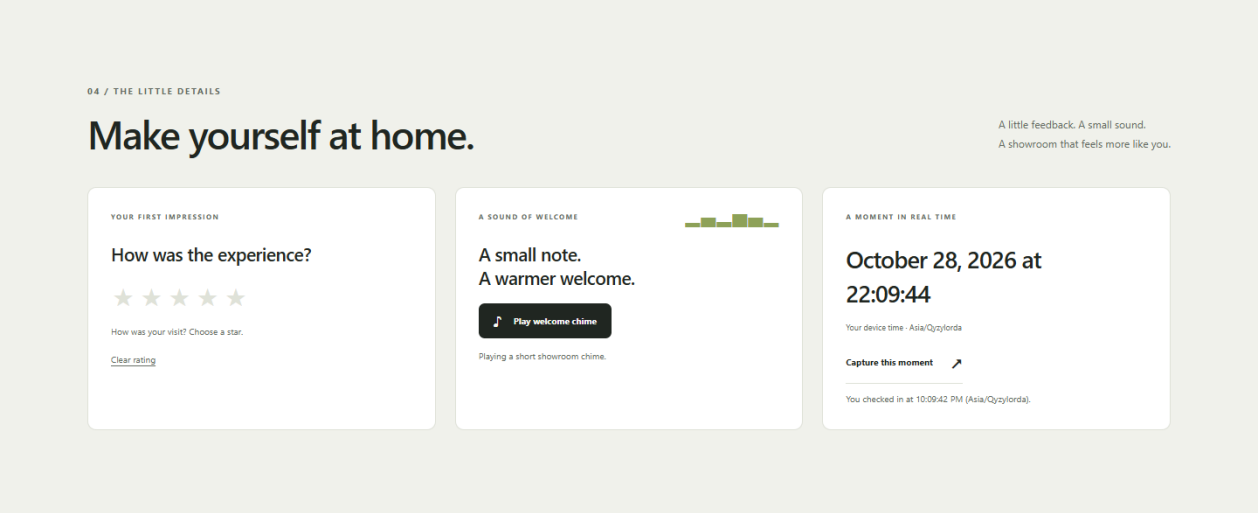


Figure 13. Sound control and interactive ownership content.

js/experience.js

```
102    async function playNotification() {
103      const request = ++soundRequest;
104      sound.pause();
105      sound.currentTime = 0;
106      try {
107        await sound.play();
108        if (request !== soundRequest) return;
109        soundButton.dataset.audioState = 'playing';
110        soundStatus.textContent = 'Playing a short showroom chime.';
111        window.Apex.animate(soundButton);
112      } catch (error) {
113        if (request !== soundRequest || error.name === 'AbortError') return;
114        soundButton.dataset.audioState = 'error';
```

js/data.js

```
 39      animate(element) {
 40        if (!element || this.prefersReducedMotion() || !element.animate) return;
 41        element.animate(
 42          [{opacity: 0, transform: 'translateY(12px)'}, {opacity: 1, transform:
    'translateY(0)'}],
 43          {duration: 300, easing: 'ease-out'}
 44        );
 45      }
 46    };
 47  })();
```

Actual source excerpt: js/experience.js, js/data.js.

09 / IMPLEMENTATION EVIDENCE

Small details, clear structure.

The FAQ, live clock and time button extend the interactive dealership experience. Structure, presentation and behavior are kept in separate files.

1. Toggle FAQ state with a click listener; CSS provides a smooth reveal and the button reports its state.
2. Create a fresh Date, format it for people, and refresh the clock every second.
3. Link one external stylesheet and four external defer scripts. Keep user-entered values out of HTML markup.

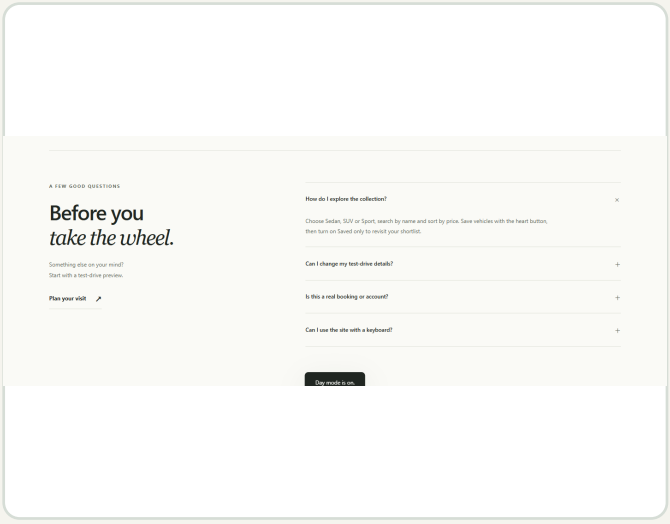


Figure 14. An FAQ answer expands on demand.

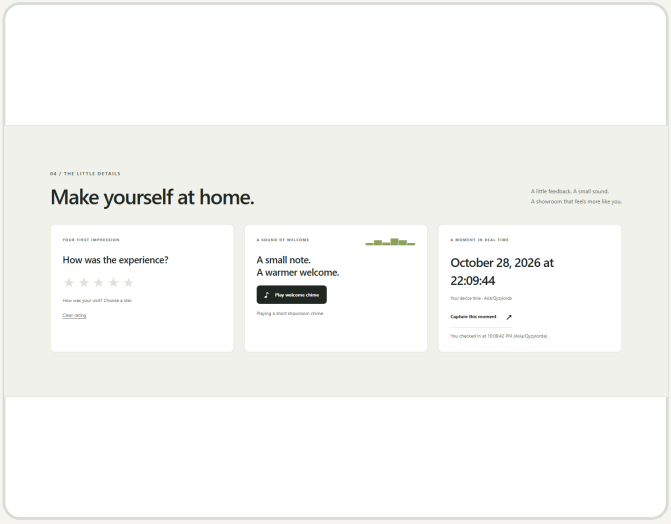


Figure 15. Live date/time and the time-check button.

js/experience.js

```
129    function updateClock() {
130      const now = new Date();
131      clock.dateTime = now.toISOString();
132      clock.textContent = now.toLocaleString('en-US', {month: 'long', day: 'numeric', year:
     'numeric', hour: '2-digit', minute: '2-digit', second: '2-digit', hour12: false});
133    }
134    updateClock();
135    setInterval(updateClock, 1000);
```

index.html

```
 14      <script src="js/data.js" defer></script>
 15      <script src="js/inventory.js" defer></script>
 16      <script src="js/experience.js" defer></script>
 17      <script src="js/booking.js" defer></script>
```

js/experience.js

```
140    document.querySelectorAll('.faq-toggle').forEach(button => {
141      const answer = document.getElementById(button.getAttribute('aria-controls'));
142      answer.setAttribute('aria-hidden', 'true');
143      answer.inert = true;
144      button.addEventListener('click', () => {
145        const open = button.getAttribute('aria-expanded') !== 'true';
146        button.setAttribute('aria-expanded', String(open));
147        button.closest('.faq-item').classList.toggle('is-open', open);
```

Actual source excerpt: js/experience.js, index.html.

REVIEW / READINESS FOR SUBMISSION

Checked, explained, ready to defend.

Verification evidence. The browser harness records its assertions in screenshots/test-results.json. It exercises actual controls and form flows; screenshots in this report are captured from the resulting pages.

Recorded verification: 141 of 141 assertions passed across 3 runs (chrome HTTP, msedge HTTP, chrome local file). 47 unique behavior checks; 0 runtime/resource errors recorded.

Deployment verified. Application commit 2f6c319 was pushed to the shared repository. GitHub Pages workflow 40733558563 completed successfully. The published URL returned HTTP 200 with the expected title. A separate live Chrome run at desktop 1440 x 1000 and mobile 390 x 844 passed 47/47 checks with 0 runtime/resource errors. Results: test-results-live.json.

DOM and collection. Category/search/sort combinations, saved state, counts, no-results/reset, gallery attributes.

Events and form. Arrow focus movement, step validation, Back retention, review, consent, confirmation and reset.

Experience and layout. Theme, rating, story, FAQ, clock, sound restart, responsive layout and runtime errors.

Challenges and solutions. Rendering filtered cards can remove the focused save button; the script restores focus to the replacement control. Multiple form steps can hide invalid fields; validation returns to the correct step and focuses the first error. Repeated sound clicks can overlap; one audio instance is stopped and restarted. Optional storage is guarded so a blocked storage policy does not prevent the page from working.

Reflection. A consistent data model keeps the displayed cards, details and booking choices aligned. Reusable rendering and validation functions make the project easier to explain and change. The booking is deliberately identified as a preview; a production service would need a real backend and server-side checks.

Defense preparation. Every student should understand all features and practise a small code change. The next page suggests preparation responsibilities; the separate Kazakh guide provides questions and a demonstration sequence.

Submission. Each member submits the report and project ZIP and attends the practice-lesson defense. The supplied LMS screenshot shows 29 October 2026, 14:00; check the LMS for the current deadline.

Website: https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/

PEOPLE / SAME THEME, SAME TEAM

Meet the team behind Apex.

The website continues the team's earlier Apex Motors identity and includes the four supplied member portraits. The footer identifies the group and all members.

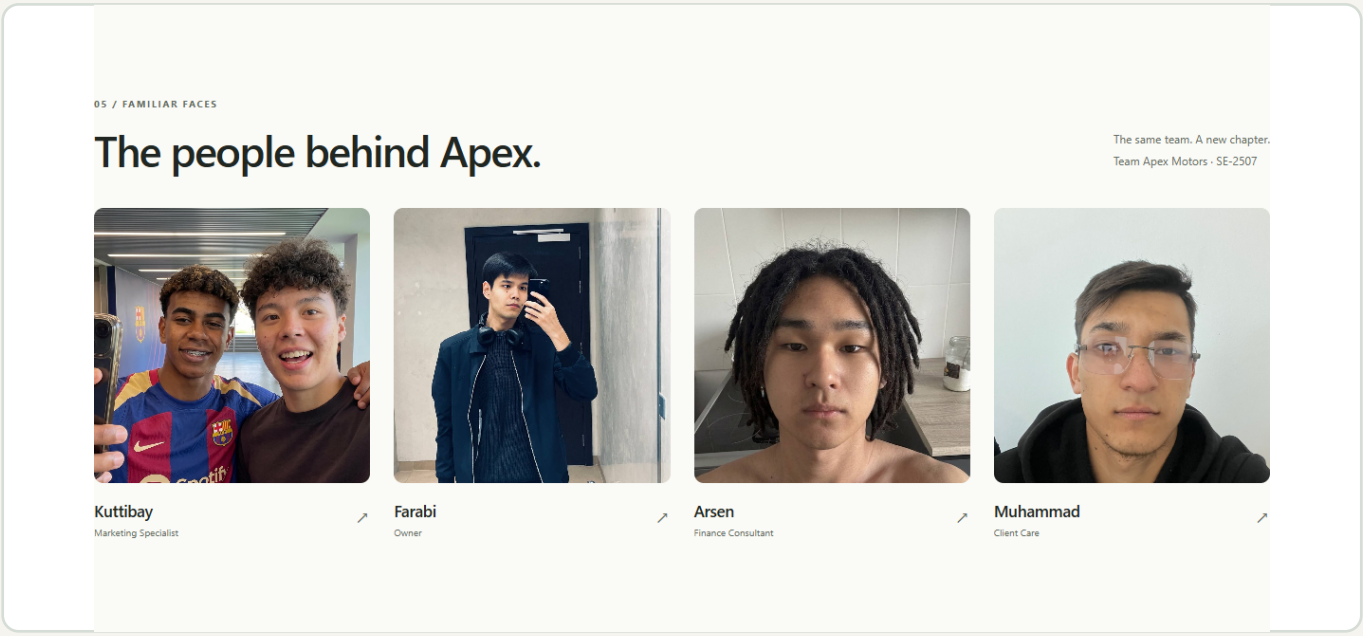


Figure 16. The team section uses the portraits supplied with the previous project.

Suggested preparation responsibilities

Arsen. Vehicle objects, dynamic inventory, category filtering and gallery attributes.

Kuttibay. Three-step form, callback navigation, validation, review and reset.

Farabi. Keyboard navigation, rating, story visibility and FAQ states.

Muhammad. Shared utilities, theme switch, sound, animation and live clock.

These are suggestions for practice, not a claim about who performed the implementation. Every member should be able to explain the whole project and answer the instructor's questions.

Before the lesson

1. Run the full interaction flow and open each JavaScript file.
2. Practise adding one vehicle, editing a filter or changing a validation rule.
3. Explain what DOM selection, a callback, an object method and a higher-order function do.
4. Submit the PDF and ZIP individually, then attend the defense.